

Module 7

I²C — Protocol + RTL + Basic Testbench

Learning objectives

- How START/STOP are detected/produced (SDA transitions while SCL high).
- How the address + R/W bit and data bytes are serialized on SDA.
- How to build a basic “bus monitor” that reconstructs bytes by sampling SDA on SCL rising edges.

Prerequisites

- Modules 1–6.
- Verilator, Make, C++ compiler. No UVM required for this baseline.

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["I²C Protocol Essentials"]

T2["RTL Architecture (Baseline)"]

T1 --> T2

T3["Basic Testbench"]

T2 --> T3

Understand the I²C protocol (start/stop, addressing, ACK/NACK), translate it to RTL (simple master + timing), and verify with a **basic** (...)

Overview

- Module 7 is the I²C **protocol + RTL + basic testbench** module (UVM comes in Module 8).

Syllabus topics

1. I²C Protocol Essentials

- Bus idle: SCL=1, SDA=1.
- START: SDA goes 1→0 while SCL is 1.
- STOP: SDA goes 0→1 while SCL is 1.
- Address phase: 7-bit address + R/W bit (0=write, 1=read) sent MSB first.
- Data phase: 8-bit data bytes sent MSB first.
- ACK/NACK (concept): receiver pulls SDA low for ACK on the 9th clock; high means NACK.

2. RTL Architecture (Baseline)

- `clk_div`: Divides ``clk`` to produce ``clk_div_tick`` used to toggle/advance SCL timing.
- `i2c_master`: State machine that generates:
 - START condition
 - address+W bits
 - one data byte
 - STOP condition

3. Basic Testbench

- Stimulus: Pulses ``start``, sets ``addr`` and ``data_in``, waits for ``done``.
- Monitor/self-check: Detects START, then samples SDA on each rising edge of SCL to reconstruct:
 - 8 bits of address+W
 - 8 bits of data

Hands-on examples

Module 7 self-check

```
$ ./scripts/module7.sh --check
```

Terminal: module7_check

```

[0:36m=====
[0:36mModule 7: Demo commands (I²C baseline)
[0:36m=====

From repo root:

# 1) Environment sanity:
verilator --version
make --version

# 2) Run the I²C baseline example (basic TB, no UVM):
cd module7/examples/i2c_baseline
make run

[1:33m[INFO] See module7/EXAMPLES.md and module7/examples/i2c_baseline/README.md for more.
```

Example 1: I²C baseline

- Drives a single write transaction (START → address+W → data → STOP)
- Includes a simple bus monitor that samples SDA on SCL rising edges to reconstruct bytes
- No UVM

Demo: I²C baseline

```
$ ./scripts/module7.sh --run
```

Terminal: ex01_i2c_baseline

The run should print:

```
- `[PASS] I2C baseline: write transaction observed correctly`  
- `I2C baseline test PASS`
```

Practice & assessment

Exercises

- Run i2c_baseline
- Trace one transfer
- Bus monitor
- Optional: Waveforms

Assessment checklist

- Can describe I²C START, STOP, address+R/W, and data phase (and ACK/NACK concept).
- Can explain the role of i2c_master and clk_div; know the baseline uses push-pull (simplified).
- Can run i2c_baseline and interpret the test result.
- Ready for Module 8: I²C UVM+SV verification.

Summary & next steps

- Understand the I²C protocol (start/stop, addressing, ACK/NACK), translate it to RTL (simple master...
- Complete module7/CHECKLIST.md
- Run `./scripts/module7.sh --check`
- Next: I²C UVM+SV